

Assignment 6: Capstone Project

Title (open book assignment) (plagism is strictly considered, do not copy paste)

Deep Learning Capstone: Advanced Application Development

Problem Statement

Design and implement a comprehensive deep learning solution that demonstrates your mastery of neural network architectures, training strategies, and real-world problem-solving. You will choose one of three project tracks, each requiring integration of multiple concepts learned throughout the course.

Project Tracks (Choose ONE)

Track A: Sentiment Analysis with Advanced NLP

Problem Statement

Build an end-to-end sentiment analysis system that classifies movie reviews, tweets, or product reviews into sentiment categories. Implement multiple architectures (RNN, LSTM, GRU, CNN for text) and compare their performance.

Dataset Options

1. **IMDB Movie Reviews:** [Stanford - IMDB Dataset](#)
 - 50,000 reviews (25k train, 25k test), binary sentiment
2. **Twitter Sentiment Analysis:** [Kaggle - Sentiment140](#)
 - 1.6 million tweets, binary sentiment
3. **Amazon Product Reviews:** [UCI - Amazon Reviews](#)
 - Multi-domain reviews with sentiment labels

Detailed Requirements

Phase 1: Data Preparation (15%)

1. Load and explore the dataset (class distribution, review length distribution)
2. Text preprocessing:
 - Lowercase conversion
 - Remove HTML tags, URLs, special characters
 - Tokenization (word-level and character-level)

- Remove stopwords (compare with/without)
- 3. Create vocabulary (top 10,000 words)
- 4. Sequence padding/truncation (max length: 200-500 words)
- 5. Word embeddings:
 - Train custom Word2Vec embeddings (gensim)
 - Use pre-trained GloVe embeddings (100d or 300d)
 - Compare both approaches

Phase 2: Model Development (40%)

Implement and compare **at least 3** of the following architectures:

1. Bidirectional LSTM

Embedding → Bidirectional LSTM(128) → Dropout(0.5) → Dense(64) → Dense(num_classes)

2. GRU with Attention

Embedding → GRU(128, return_sequences=True) → Attention Layer → Dense(num_classes)

3. 1D CNN for Text

Embedding → Conv1D(128, 5) → MaxPool → Conv1D(64, 3) → GlobalMaxPool → Dense(num_classes)

4. Hybrid CNN-LSTM

Embedding → Conv1D(128, 5) → MaxPool → LSTM(64) → Dense(num_classes)

5. Transformer-based (Advanced)

- Fine-tune BERT, DistilBERT, or RoBERTa
- Use HuggingFace Transformers library

Phase 3: Training & Optimization (20%)

1. Implement proper train/validation/test split (70/15/15)
2. Use class weights if data is imbalanced
3. Implement learning rate scheduling
4. Apply regularization (dropout, L2)
5. Track metrics: Accuracy, Precision, Recall, F1-score
6. Save best model based on validation F1

Phase 4: Advanced Features (15%)

1. **Interpretability:** Implement attention visualization or LIME
2. **Error Analysis:** Analyze misclassified samples

3. **Ensemble:** Combine predictions from top 3 models
4. **Real-time Demo:** Build Streamlit/Gradio interface for live sentiment prediction

Phase 5: Deployment (10%)

1. Create REST API using Flask/FastAPI
2. Dockerize the application
3. Deploy to cloud (Heroku, AWS, or Google Cloud) - optional
4. Create user-friendly interface

Track B: Multi-Class Image Classification Challenge

Problem Statement

Develop a state-of-the-art image classification system for a challenging real-world dataset. Achieve maximum accuracy through architectural innovations, transfer learning, and ensemble methods.

Dataset Options

1. **Food-101:** [ETH Zurich - Food-101](#)
 - 101 food categories, 101,000 images
2. **Stanford Dogs:** [Stanford Dogs Dataset](#)
 - 120 dog breeds, 20,580 images
3. **Caltech-256:** [Caltech-256](#)
 - 256 object categories, 30,607 images

Detailed Requirements

Phase 1: Data Pipeline (15%)

1. Implement efficient data loading with PyTorch DataLoader or tf.data
2. Comprehensive data augmentation:
 - Random crops, flips, rotations
 - Color jittering, brightness/contrast adjustment
 - Cutout or Mixup augmentation
 - Auto-augmentation (optional)
3. Handle class imbalance:
 - Weighted sampling
 - Class weights in loss function

4. Split strategy: Stratified split maintaining class distribution

Phase 2: Model Architecture (40%)

Implement **at least 3** different approaches:

1. Custom CNN Architecture

- Design from scratch with modern techniques:
- Residual connections (ResNet-inspired)
- Squeeze-and-Excitation blocks
- Depthwise separable convolutions (MobileNet-inspired)
- Batch normalization and dropout

2. Transfer Learning with Fine-tuning

- Use pre-trained models: ResNet50, EfficientNet, Vision Transformer
- Strategy A: Freeze early layers, train final layers
- Strategy B: Full fine-tuning with low learning rate
- Strategy C: Gradual unfreezing (progressive fine-tuning)

3. Ensemble Methods

- Train 3-5 different models
- Implement voting (hard/soft)
- Implement stacking with meta-learner
- Test-time augmentation (TTA)

Phase 3: Advanced Training (20%)

1. Implement cosine annealing or OneCycle learning rate schedule
2. Use mixed precision training for efficiency
3. Progressive image resizing (start small, increase size)
4. Knowledge distillation (optional): Train large teacher → small student
5. Multi-scale testing

Phase 4: Analysis & Interpretability (15%)

1. Generate confusion matrix and per-class metrics
2. Implement GradCAM for visual explanations
3. Analyze failure cases systematically
4. Create class activation maps for misclassifications
5. t-SNE visualization of final layer embeddings

Phase 5: Optimization & Deployment (10%)

1. Model compression (pruning or quantization)
2. Measure inference time and model size
3. Create production-ready pipeline
4. Build interactive demo (Gradio or Streamlit)

Track C: Hybrid Autoencoder + Classifier System

Problem Statement

Design a two-stage deep learning system that first learns useful representations using autoencoders, then uses these representations for classification. Demonstrate that learned features improve classification performance.

Dataset Options

1. **Fashion-MNIST:** [Fashion-MNIST](#)
 - 70,000 grayscale images, 10 fashion categories
2. **CIFAR-100:** [CIFAR-100](#)
 - 60,000 color images, 100 fine-grained classes
3. **Chest X-Ray Pneumonia:** [Kaggle - Chest X-Ray](#)
 - Medical imaging, binary classification (normal/pneumonia)

Detailed Requirements

Phase 1: Autoencoder Pre-training (25%)

1. **Implement Multiple Autoencoder Types:**
 - Vanilla autoencoder (baseline)
 - Convolutional autoencoder
 - Denoising autoencoder
 - Variational autoencoder (VAE)
 - Contractive autoencoder (advanced)
2. **Feature Learning:**
 - Train each autoencoder unsupervised
 - Experiment with different bottleneck dimensions (16, 32, 64, 128)

- Visualize learned representations
- Evaluate reconstruction quality

3. **Feature Extraction:**

- Extract bottleneck features for entire dataset
- Visualize latent space using t-SNE/UMAP
- Analyze feature separability

Phase 2: Classification Pipeline (30%)

1. **Baseline Classifier:**

- Train simple CNN/MLP on raw pixels
- Establish baseline accuracy

2. **Autoencoder-based Classifiers:**

- **Approach A:** Freeze encoder, train classifier on latent features
- **Approach B:** Fine-tune encoder + classifier end-to-end
- **Approach C:** Use autoencoder as regularizer (auxiliary task)

3. **Architecture:**

Frozen Encoder → Latent Features → Dense(256) → Dropout → Dense(num_classes)

4. **Compare Performance:**

- Raw pixels vs. learned features
- Different autoencoder types
- Different latent dimensions

Phase 3: Advanced Techniques (20%)

1. **Semi-Supervised Learning:**

- Use only 10% of labeled data
- Pre-train on all data (unlabeled)
- Fine-tune on labeled subset
- Compare with supervised-only baseline

2. **Adversarial Training:**

- Implement adversarial autoencoder (AAE)
- Use discriminator to enforce latent space distribution

- Evaluate if it improves classification

3. **Multi-Task Learning:**

- Joint training: reconstruction + classification
- Weighted loss: $\alpha \times \text{reconstruction_loss} + \beta \times \text{classification_loss}$
- Experiment with α, β values

Phase 4: Anomaly Detection (15%)

1. Use trained autoencoder for anomaly detection
2. Identify samples with high reconstruction error
3. Set threshold for anomaly classification
4. Evaluate on corrupted/out-of-distribution samples
5. Create anomaly score distribution visualization

Phase 5: Comprehensive Analysis (10%)

1. **Feature Quality Analysis:**

- Measure feature separability (Silhouette score)
- Compare PCA vs. autoencoder features
- Analyze which autoencoder learns best features

2. **Ablation Studies:**

- Effect of bottleneck size
- Impact of denoising pre-training
- Benefits of VAE regularization

3. **Visualization Suite:**

- Latent space interpolation
- Feature importance via gradient analysis
- Reconstruction quality across classes

Common Requirements for All Tracks (Applies to A, B, C) (you need to choose any one of above project for capestone)

Code Quality (10%)

1. Well-structured, modular code
2. Proper documentation and comments
3. Configuration files for hyperparameters
4. Requirements.txt and setup instructions
5. Git repository with meaningful commits

Reproducibility (5%)

1. Set random seeds for reproducibility
2. Document all hyperparameters
3. Save training logs and checkpoints
4. Include environment specifications

Documentation (10%)

1. **README.md:**
 - Project overview and objectives
 - Setup instructions
 - How to run training and inference
 - Results summary
2. **Technical Report** (10-15 pages):
 - Introduction and motivation
 - Literature review (3-5 relevant papers)
 - Methodology and architecture details
 - Experimental setup and hyperparameters
 - Results with tables and visualizations
 - Ablation studies and comparisons
 - Error analysis and limitations
 - Conclusion and future work
 - References

3. Presentation Slides (15-20 slides):

- Problem statement
- Approach and innovations
- Key results and visualizations
- Demo (if applicable)
- Conclusions

Demonstration (5%)

1. Create video demo (3-5 minutes) or live demo
2. Show end-to-end workflow
3. Demonstrate real-time predictions
4. Explain key features and results

Expected Deliverables (All Tracks)

1. Code Repository

- Complete source code (organized in folders)
- Trained model weights (via Google Drive/Dropbox link)
- Jupyter notebooks with experiments
- Requirements.txt
- README.md

2. Models

- All trained models (best checkpoints)
- Training logs and metrics history
- Experiment tracking (use Weights & Biases or TensorBoard)

3. Visualizations (minimum 20)

- Training/validation curves
- Confusion matrices
- Performance comparison charts
- Architecture diagrams
- Sample predictions/results
- Error analysis visualizations
- Attention maps / GradCAM (if applicable)
- Latent space visualizations (Track C)

4. Technical Report (PDF)

- 10-15 pages, IEEE or ACM format
- Include all sections mentioned above
- High-quality figures and tables

5. Presentation Materials

- PowerPoint/PDF slides
- Video demo or live demonstration script

6. Supplementary Materials

- Ablation study results
- Hyperparameter search logs
- Dataset statistics and analysis

Evaluation Rubric (100 points)

Criteria	Points	Description
Technical Implementation	30	Correct architecture, proper training, multiple models implemented
Performance	20	Meets or exceeds benchmark accuracy, quality results
Innovation & Creativity	15	Novel approaches, clever solutions, advanced techniques
Experimentation	10	Thorough ablations, comparisons, hyperparameter tuning
Code Quality	10	Clean, modular, documented, reproducible
Report & Documentation	10	Comprehensive, well-written, professional
Visualizations	5	Clear, informative, publication-quality

Bonus Points (20)

- Achieve top 10% performance on benchmark (+5)
- Implement novel architecture or technique (+5)
- Deploy as production-ready application (+5)
- Open-source contribution quality (GitHub stars, documentation) (+3)
- Present at student research symposium (+2)

Target Metrics by Track

Track A (Sentiment Analysis)

- Accuracy: >88% (good), >92% (excellent)
- F1-Score: >0.87 (good), >0.91 (excellent)
- Inference time: <100ms per sample

Track B (Image Classification)

- Top-1 Accuracy: >75% (good), >82% (excellent) [depends on dataset]
- Top-5 Accuracy: >92%
- Model size: <50MB (deployable)

Track C (Hybrid System)

- Classification accuracy improvement over baseline: >5% (good), >10% (excellent)
- Reconstruction MSE: <0.02
- Anomaly detection AUC: >0.85

Findings/Observations to Document (All Tracks)

1. What architectural choices had the most impact on performance?
2. How did different regularization techniques affect generalization?
3. What patterns emerged in the error analysis?
4. How do different training strategies compare (learning rate schedules, optimizers)?
5. What are the computational trade-offs (accuracy vs. speed vs. model size)?
6. How well does the model generalize to out-of-distribution samples?
7. What are the practical limitations and failure cases?
8. What would you do differently given more time/resources?
9. How does your approach compare to state-of-the-art methods?
10. What are the ethical considerations for deploying this model?
